



Informe Final del Proyecto

Versión: 1.0.0



CloudMetric

15 DE ENERO DE 2026

Índice

1. Resumen Ejecutivo	2
2. Arquitectura y Stack Tecnológico	2
2.1. Nivel de Hardware (Sensorización)	2
2.2. Nivel de Backend (Lógica y Datos)	2
2.3. Nivel de Frontend Web (Panel de Control)	2
2.4. Nivel de Aplicación Móvil (Android)	3
3. Desglose de Entregas por Sprint	3
4. Informe Económico	7
4.1. Desglose de Costes por Módulo	7
4.2. Costes de Infraestructura y Materiales (Estimado)	8
4.3. Resumen Financiero Total	8
5. Conclusiones	8

1. Resumen Ejecutivo

El objetivo principal de este proyecto ha sido el desarrollo de una solución integral IoT para la monitorización ambiental. El sistema permite la recolección de datos biométricos mediante nodos sensores basados en Arduino, la transmisión de estos datos vía Bluetooth Low Energy (iBeacon), y su posterior procesamiento y visualización en plataformas Web y Móvil.

La solución abarca desde el hardware embebido hasta la nube, proporcionando herramientas para ciudadanos (visualización de calidad del aire) y administradores (gestión de incidencias y usuarios).

2. Arquitectura y Stack Tecnológico

El sistema se ha construido siguiendo una arquitectura distribuida y escalable en la nube.

2.1. Nivel de Hardware (Sensorización)

- **Dispositivo:** Microcontrolador compatible con Arduino.
- **Protocolo:** Bluetooth Low Energy (BLE) utilizando el estándar iBeacon.
- **Firmware:** C++ (src/HolaMundoIBeacon).
- **Funcionalidad:** Lectura de sensores simulados/reales y broadcast de tramas de datos.

2.2. Nivel de Backend (Lógica y Datos)

- **Plataforma:** Google Firebase.
- **Base de Datos:** Firestore (NoSQL) para almacenamiento en tiempo real.
- **API:** Firebase Cloud Functions (Node.js).
- **Estructura:**
 - ServidorRest.js: Endpoints RESTful.
 - LogicaDeNegocio.js: Reglas de negocio y validaciones.

2.3. Nivel de Frontend Web (Panel de Control)

- **Framework:** React JS (src/WEB/my-app).
- **Estilos:** CSS3 personalizado y diseño responsive.

- **Despliegue:** Firebase Hosting.
- **Funcionalidades:** Dashboard de métricas, mapas de calor, gestión de usuarios (Admin) e intranet.

2.4. Nivel de Aplicación Móvil (Android)

- **Plataforma:** Android Nativo (Java/Kotlin).
 - **Herramientas:** Android Studio, Gradle.
 - **Funcionalidades:** Escaneo de iBeacons en segundo plano, integración con cámara para códigos QR, visualización móvil de datos.
-

3. Desglose de Entregas por Sprint

Las funcionalidades se han desarrollado de forma incremental en 4 Sprints. A continuación, se detalla la implementación técnica de cada entrega.

Sprint 1: Fundamentos, Diseño y Acceso

Objetivo: Definición del sistema, diseño y gestión básica de usuarios y datos.

- **Definición modelo de negocio y Diseño BBDD:**
 - *Implementación:* Se diseñó un esquema NoSQL en **Firestore**. Las colecciones principales son usuarios (datos de perfil y rol), nodos (sensores registrados) y lecturas (series temporales de datos ambientales). Se definieron índices compuestos para optimizar consultas por fecha y geolocalización.
- **Diseño UX/UI (Web y App):**
 - *Implementación:* Creación de prototipos de alta fidelidad y definición de la hoja de estilos global (src/WEB/my-app/src/css/main.css) siguiendo principios de diseño Responsive y Material Design.
- **Acceso a la plataforma (Login/Registro):**
 - *Implementación:* Integración del SDK firebase/auth. En Web (Login.js, Registro.js), se gestiona el estado de sesión mediante observadores onAuthStateChanged. En Android, se utiliza la API nativa de Google Sign-In y Auth con correo/contraseña.
- **Editar perfil de usuario:**

- *Implementación:* Componente React Perfil.js que utiliza useState para el manejo de formularios y updateDoc de Firestore para persistir cambios como nombre o contraseña.
- **Vincular sensor (Lectura QR):**
 - *Implementación:* En Android, uso de la librería **ZXing** o la API de Cámara nativa para decodificar códigos QR que contienen el UUID único del sensor. Este ID se asocia al documento del usuario en Firestore mediante una transacción.
- **Implementación Base de Datos:**
 - *Implementación:* Configuración del proyecto en Firebase Console. Se implementaron reglas de seguridad (firestore.rules) para restringir la escritura de lecturas solo a usuarios autenticados propietarios del nodo.
- **Notificaciones límite excedido (Base):**
 - *Implementación:* Lógica inicial en el método GuardarLecturas de LogicaDeNegocio.js. Si una lectura de CO2 supera el umbral (ej. 1000 ppm), se dispara una función que prepara una alerta.

Sprint 2: Lógica de Negocio y Gamificación

Objetivo: Aportar valor al usuario mediante incentivos y datos procesados.

- **Notificaciones estado del nodo:**
 - *Implementación:* En Android, un Service en segundo plano monitoriza el ScanCallback. Si no se reciben paquetes de publicidad del UUID registrado en X minutos, se genera una notificación local NotificationCompat.
- **Estimar distancia recorrida:**
 - *Implementación:* Uso del locationManager de Android para obtener actualizaciones GPS. Se aplica la fórmula de Haversine (_haversineDistance en el backend) para calcular incrementos de distancia entre puntos de medición sucesivos.
- **Sistema de Incentivos (Gamificación):**
 - *Implementación:* Lógica encapsulada en Cloud Functions. Se asignan "monedas" en el campo monedas del documento usuarios basándose en el tiempo de conexión activa. La tienda (Tienda.js) consulta este saldo en tiempo real.

- **Información adicional (Consejos):**

- *Implementación:* Componente estático Informacion.js en React que renderiza contenido educativo JSON, optimizado para carga rápida (Lazy Loading).

Sprint 3: Visualización Avanzada y Hardware

Objetivo: Mapas de calor, robustez del sensor y biometría.

- **Facilitar localización del sensor:**

- *Implementación:* En la App Android, se lee el valor **RSSI** (Received Signal Strength Indicator) del paquete BLE. Se mapea este valor (dBm) a una escala visual (colores/barras) para guiar al usuario hacia el dispositivo.

- **Diseño de Carcasa y Robustez:**

- *Implementación:* Modelado 3D de una caja protectora ventilada. El firmware Arduino (HolaMundoIBeacon.ino) implementa un Watchdog Timer para reiniciarse automáticamente en caso de bloqueo físico o eléctrico.

- **Acceso Biométrico:**

- *Implementación:* Integración de la librería androidx.biometric. Se utiliza BiometricPrompt para solicitar huella dactilar o reconocimiento facial antes de conceder acceso al token de sesión de Firebase.

- **Mapas de contaminación (3 niveles/colores):**

- *Implementación:* Uso de librerías de mapas (como Leaflet o Google Maps API). Los marcadores se colorean dinámicamente según el valor del sensor: Verde (<600ppm), Amarillo (600-1000ppm), Rojo (>1000ppm).

- **Búsqueda Geoespacial Eficiente (Geohash):**

- *Implementación:* En LogicaDeNegocio.js (método obtenerLecturas), se utiliza la librería geofire-common. Se calcula un **geohash** para cada lectura al guardarla. Para consultar, se calculan los "bounds" del geohash según el radio de búsqueda y se ejecutan consultas paralelas (Promise.all) a Firestore, filtrando luego por distancia exacta en memoria.

- **Calidad general del aire (ICA):**

- *Implementación:* Algoritmo backend que normaliza los valores de diferentes sensores (CO2, NO2, Temperatura) a una escala común (0-100) y devuelve el valor máximo como índice de calidad predominante.

Sprint 4: Gestión, Incidencias y Cierre

Objetivo: Herramientas administrativas, mantenimiento y entrega final.

- **Firmware Arduino Optimizado:**

- *Implementación:* Código C++ (HolaMundoIBeacon.ino) utilizando la librería bluefruit.h de Adafruit. Se configura el Publicador para emitir tramas iBeacon estándar (UUID, Major, Minor, TxPower). Se implementan ciclos de *sleep* para reducir consumo.

- **Gestión de Incidencias (CRUD Completo):**

- *Implementación:*
 - **Backend:** Colección incidencias en Firestore. Métodos en LogicaDeNegocio.js (reportarIncidencia, resolverIncidencia) que gestionan estados ("pendiente", "resuelta") y asignación de administradores.
 - **Frontend:** Componente Incidencias.js con formularios controlados. Lista incidencias filtrando por usuario. Uso de batch writes para actualizaciones atómicas si fuera necesario.

- **Manual de Usuario:**

- *Implementación:* Documentación técnica generada en PDF, incluyendo diagramas de flujo y capturas de pantalla de los flujos de la aplicación.

- **Informe estado de nodos (Admin):**

- *Implementación:* Función administrativa obtenerNodosDesdeAdmin que cruza datos de las colecciones usuarios, nodos y la última lectura de lecturas. Calcula campos derivados como activo24h comparando el timestamp de la última lectura con Date.now().

- **Histórico de mapas:**

- *Implementación:* Scheduled Cloud Function (Cron job) que ejecuta diariamente una agregación de las lecturas del día, calcula

promedios por zona (geohash) y guarda el resultado en una colección historico_mapas optimizada para lectura rápida.

4. Informe Económico

El cálculo de costes se basa en horas de ingeniería estimadas y costes de infraestructura iniciales. Se asume una tarifa media de desarrollo de **45 €/hora**.

4.1. Desglose de Costes por Módulo

<i>Módulo</i>	<i>Funcionalidad Clave</i>	<i>Horas Est.</i>	<i>Coste (€)</i>
<i>Hardware (Arduino)</i>	Programación Firmware BLE & Drivers	25h	1.125 €
	Diseño de circuito y prototipado	10h	450 €
<i>Backend (Firebase)</i>	API REST & Cloud Functions	35h	1.575 €
	Diseño BBDD Firestore & Auth	15h	675 €
<i>Frontend (Web)</i>	Dashboard, Gráficas y Mapas	40h	1.800 €
	Panel de Administración & CRUD	25h	1.125 €
<i>Móvil (Android)</i>	Escáner BLE & Lógica de sensores	45h	2.025 €
	Interfaz UI/UX & Integración QR	25h	1.125 €
<i>Gestión</i>	Documentación, Diseño (Draw.io) y Tests	30h	1.350 €
<i>TOTAL DESARROLLO</i>		250h	11.250 €

4.2. Costes de Infraestructura y Materiales (Estimado)

Concepto	Detalle	Coste
Hardware	Kit Arduino, Sensores CO2/Temp, Módulos BLE	30 €
Servicios Cloud	Firebase (Plan Spark - Gratuito durante desarrollo)	0 €
Licencias	Software de desarrollo (IDEs Community, Open Source)	0 €
TOTAL MATERIALES		30 €

4.3. Resumen Financiero Total

Concepto	Importe Total
<i>Desarrollo de Software e Ingeniería</i>	11.250 €
<i>Hardware y Materiales</i>	30 €
PRESUPUESTO TOTAL DEL PROYECTO	11.280 €

5. Conclusiones

El proyecto ha cumplido con los objetivos, entregando un producto funcional que conecta el mundo físico con el digital.